

Technical Walkthrough: Simulated Mini-SOC Pipeline

1. Design Rationale

For this capstone, I built a small simulated SOC pipeline that shows how an AI-supported detection and response workflow could operate without touching real systems. The project uses AI-generated synthetic Suricata-style telemetry, not live traffic, real malware, production firewall rules, endpoint isolation, or a production SIEM or SOAR platform. That safety boundary is intentional. I wanted the prototype to be realistic enough to explain a SOC workflow, but limited enough that it stays appropriate for a class assignment.

The basic idea is that a security event should not jump straight from "the model thinks this is bad" to an automated response. The pipeline processes events in stages: intake, feature extraction, classification, risk scoring, deterministic safety checks, response routing, and evidence preservation. This makes the system easier to explain and easier to audit. If something looks malformed, evasive, or uncertain, the workflow fails into review instead of trusting the classifier blindly.

I chose a scikit-learn decision tree as the model because it is simple and explainable. A more complex model might score better in a real SOC, but that was not the goal here. The goal was to demonstrate a bounded semi-autonomous agent that can classify events, assign risk, and route safe simulated actions while preserving a decision trail. The decision tree is trained in memory each run from the combined synthetic dataset, so there is no saved model artifact. The evidence log records this honestly as `model_sha256: "in_memory_not_persisted"`.

The response logic is also deliberately modest. The system can auto-close low-risk benign events, create JSON tickets, send uncertain cases to a review queue, and add high-confidence malicious source IPs to a simulated blocklist text file. The blocklist is only a local file. It does not connect to a real firewall or change a real network control.

2. Architecture Diagram

The implemented workflow follows this structure:

AI-generated synthetic Suricata-style events

- > Event intake and validation
- > Feature extraction
- > Decision-tree classifier
- > Risk scoring
- > Deterministic safety checks
- > Response routing
- > Evidence decision log
- > Forensic export package

Safety and response branches:

- > Human review queue
- > Simulated incident ticket
- > Simulated IP blocklist

The main end-to-end command is:

```
python -m minisoc.cli respond data\sample\demo_scenario_events_20.eve.jsonl --train data\raw\synthetic_minisoc
```

That command trains the in-memory classifier, processes the demo events, applies safety checks, writes response artifacts, and creates the decision log. A separate export command can then package evidence for one selected event:

```
python -m minisoc.cli export-evidence --event-id demo-0008 --events data\sample\demo_scenario_events_20.eve
```

In the verified demo run, the pipeline processed 20 demo records. It auto-closed 5 events, created 8 tickets, routed 3 events to review, and wrote 4 entries to the simulated blocklist. Those counts are useful in the demo because they show that the pipeline does not treat every event the same way.

3. Model Or Logic Explanation

The first stage is event intake. The intake code reads line-delimited JSON records that resemble Suricata `eve.json` events. It validates required fields such as `event_id`, `timestamp`, `event_type`, `src_ip`, `dest_ip`, `asset_id`, `expected_label`,

and the synthetic ground-truth disclosure. It also calculates a SHA-256 hash of the raw JSON line. That raw hash matters because it gives the later evidence log a stable reference back to the original event.

After intake, the feature extractor turns the event into a small set of numeric signals. Examples include alert severity, destination port, failed login count, internal probe count, callback interval, domain entropy, URI entropy, byte and packet ratios, asset criticality, missing field flags, obfuscation flags, and prompt-injection-like text flags. The feature extractor also produces reason codes, so the result is not just a number vector.

The classifier is a bounded decision tree trained from the combined synthetic dataset. It predicts one of six labels: `benign`, `credential_access`, `lateral_movement`, `command_and_control`, `exploit_attempt`, or `needs_human_review`. The classification result includes the predicted label, confidence score, model name, model version, feature names, and reason codes.

Risk scoring happens after classification. The score combines model confidence, alert severity, asset criticality, activity patterns, and suspicious-input indicators. The output is a numeric risk score from 0 to 100 plus a severity lane such as low, medium, or high. For example, a high-confidence credential-access event with many failed logins receives a higher score than a normal benign web event. Evasive or incomplete inputs also push the score upward because they create operational uncertainty.

The deterministic safety checks sit between the model and the response action. This is the part I would describe as the review gate. It blocks automation when the classifier is uncertain, when the event is labelled as needing human review, when intake metadata says review is required, when fields are missing, when obfuscation indicators are present, when prompt-injection-like text appears in an untrusted field, or when a benign prediction conflicts with a high risk score. This is an important design choice because the classifier helps interpret events, but it does not get final authority over automation.

Finally, response routing chooses a safe simulated action. Low-risk benign events can be auto-closed. Medium and high risk events can create tickets. Suspicious or unclear events can go to the review queue. High-confidence command-and-control or exploit-like events can write the source IP to the simulated blocklist if the safety checks allow automation.

4. Bypass And Failure Handling

The demo dataset includes suspicious edge cases so the pipeline can show more than a happy path. These include missing analysis fields, obfuscated values, high-entropy strings, and prompt-injection-like text placed inside log fields. The system treats those fields as data, not instructions.

For a malformed or incomplete event, intake and feature extraction mark missing fields. Risk scoring raises the priority, and the review gate blocks automation. For obfuscated values, the feature extractor counts obfuscation flags, and the review gate routes the event to human review. For prompt-injection-like text, the system does not follow or interpret the text as an instruction. It flags the record and sends it to review.

This behavior is important because a real SOC pipeline would see messy input. Logs can be incomplete, attackers can try to hide indicators, and security tools can disagree. In this project, the safe default is to preserve the event and ask for analyst review rather than auto-closing it or applying a simulated block just because the classifier produced a label.

5. Forensic Readiness Strategy

The pipeline preserves evidence at several points. Intake stores the raw event hash. The response workflow writes `outputs/decision_logs/decisions.jsonl`, which records the event ID, timestamp, correlation ID, community ID, raw event SHA-256, model name, model version, model hash status, feature summary, predicted label, confidence, risk score, severity, reason codes, safety-check flags, action taken, analyst override field, review status, and simulated-only flag.

The response artifacts are also saved locally. Review items are written to `outputs/review_queue/review_queue.jsonl`. Ticket records are written as JSON files under `outputs/tickets/`. Simulated block actions are written to `outputs/simulated_blocklist.txt`. These artifacts are intentionally simple, but they show what happened and where the event was routed.

The `export-evidence` command adds a small forensic package for one selected event. It creates `raw_event.json`, `decision_record.json`, `model_metadata.json`, `action_record.json`, `analyst_summary.md`, and `hash_manifest.json`

under `outputs/evidence_exports/<event_id>/`. The hash manifest stores SHA-256 hashes for the exported artifacts so an analyst or reviewer can check whether the package changed after export.

This is not a real chain-of-custody system, and I would not present it as one. It is a local forensic readiness demonstration. It shows the kind of evidence that should be preserved before a security decision is forgotten, overwritten, or explained only from memory.

6. Known Limitations

This prototype is intentionally limited. The telemetry is AI-generated synthetic data, so the evaluation shows whether the model learned the simulated patterns, not whether it would perform well in a real SOC. The project does not ingest live traffic, run Suricata, parse real PCAPs, process malware, or connect to production security tooling.

The response actions are also simulated. A blocklist entry is a text file, not a firewall rule. A ticket is a JSON artifact, not an integration with a ticketing platform. A review queue is a JSONL file, not an analyst console. This keeps the project safe, but it also means the operational parts are demonstrations rather than deployable controls.

The model is deliberately simple. It is a decision tree trained in memory from `data\raw\synthetic_minisoc_events_combined_220`. There is no persisted model artifact, no drift monitoring, no retraining pipeline, and no live feedback loop from analyst decisions. The project also does not use an LLM in the baseline version, which avoids prompt-handling risk but means there is no natural-language enrichment beyond the generated analyst summary in the export package.

Overall, I would describe the project as a safe, explainable mini-SOC prototype. It demonstrates the required pieces: event intake, threat classification, risk scoring, bypass handling, simulated response, evidence logging, and forensic export. It is not production ready, and it is not trying to be. The useful part is that each decision leaves enough evidence behind for a reviewer to understand what happened and why.

Integrity Statement

I confirm that this submission is my original work. Any external tools, references, datasets, frameworks, or code libraries used in this project have been acknowledged. I understand that the work must represent my own design, analysis, and implementation decisions.